

Суть проекта.

ColorModels – учебный проект студентов 4 курса по изучению цветовых моделей изображений

Классы и структуры.

task.h

AppContext – класс единого контекста, который получает любая задача при её выполнении и который содержит всю вспомогательную информацию

TaskInterface – интерфейс задачи, содержащий только 3 метода, принимающих на вход AppContext: prepare (выполнение задачи с тяжёлыми вычислениями, выполняющееся один раз при первом открытии задачи), drawByTexture (отрисовка результата задачи при помощи текстур), drawByPixels (отрисовка результата задачи по-пиксельно)

Task1 : TaskInterface – класс задачи 1

Task2 : TaskInterface – класс задачи 2

Task3 : TaskInterface – класс задачи 3

images.h

PixelHSV – тройка значения HSV

Image – базовый класс изображения, содержащий 3 поля: width, height, channels –, а также 2 метода: empty() и pixelNumber()

ImageRGB : Image – класс изображения в RGB-модели, содержащий вектор байтов data вида

ImageGray : Image – класс полутонового изображения, , содержащий вектор байтов data вида

ImageHSV : Image – класс изображения в HSV-модели, содержащий вектор data троек значений HSV типа PixelHSV

Все эти классы-наследники содержат метод поэлементного обращения at() с собственной структурой реализации.

image_processing.h

loadImageRGB() – загрузка изображения в RGB-модели по указанному пути

RGBtoGRAY() – преобразование изображения RGB-модели в полутоновое изображение

RGBtoHSV() – преобразование изображения RGB-модели в изображение HSV-модели

diffGray() – вычисление разницы между двумя полутоновыми изображениями

RGBconvertHSV() – преобразование RGB-тройки в HSV-тройку

HSVconvertRGB() – преобразование HSV-тройки в RGB-тройку

`applyHSV()` – применение цветокоррекции в пространстве HSV к изображению HSV-модели

`intensityHistogram()` – подсчёт гистограммы яркости полутонового изображения

`drawHistogramByTexture()` – отрисовка гистограммы изображения при помощи текстуры

`makeTextureRGB()` – создание `SDL_Texture` на основе изображения RGB-модели

`makeTextureGray()` – создание `SDL_Texture` на основе полутонового изображения

`saveImagePNG()` – сохранение изображения в формате `.png` по указанному пути

`drawImageRGBByPixels()` – отрисовка изображения RGB-модели по-пиксельно

`downscale()` – уменьшение изображения RGB-модели с отбрасыванием части пикселей (для быстрых вычислений)

dialog_windows.h

Жизненный цикл

1. Инициализация критических компонентов: `SDL_window* window` (указатель на окно), `SDL_Renderer* renderer` (рендерер для отрисовки компонент), `SDL_Event eventer` (контролёр событий), `SDL_Init(SDL_INIT_VIDEO)` (вызов SDL-подсистемы с указанными флагами), `IMGUI_CHECKVERSION()` (проверки версий файлов `.h` библиотеки `ImGui`), `ImGui::CreateContext()` (создание глобального контекста, в котором содержится очередь окон и виджетов, состояния вывода, стили и т.п.), `ImGui_ImplSDLRenderer3_Init(renderer)` (вызов слоя, отвечающего за отрисовку в `ImGui`), `ImGui::GetIO()` (получение глобального объекта ввода-вывода)
2. Инициализация вспомогательных переменных: `ImageRGB image` (загруженное изображение в RAM), `SDL_Texture* texImage = nullptr` (SDL-текстуры загруженного изображения – нужно для текстурного отображения), `TaskInterface* currentTask = nullptr` (указатель на текущую активную задачу), `ActiveTask activeTask = ActiveTask::None` (текущая активная задача)
3. Запуск главного цикла: `while (running)` – цикл, в каждой итерации которого формируется один кадр приложения
4. Обработка очереди событий: `while (SDL_PollEvent(&eventer))` – извлечение всех накопившихся событий SDL из очереди и их обработка
5. Обработка результата диалогового окна выбора файла и его сохранения
6. Начало кадра `ImGui` – последовательно вызываются `ImGui_ImplSDLRenderer3_NewFrame()` (подготовка внутренних буферов рендера `ImGui`), `ImGui_ImplSDL3_NewFrame()` (передача накопленных событий ввода и актуализация состояния мыши/клавиатуры), `ImGui::NewFrame()` (создание нового кадра, в течение которого формируется список виджетов)
7. Отрисовка главного окна с заданием чётких позиций и размеров

8. Отрисовка исходного изображения в левой области: если image не пустое и texImage создана, вычисляется коэффициент масштабирования $scale = \min\left(\frac{avail.x}{image.width}, \frac{avail.y}{image.height}\right)$, и картинка вписывается в доступную область с сохранением пропорций
9. Отрисовка кнопок в правой области. Кнопки "Task 1", "Task 2", "Task 3" удаляют текущую задачу (delete currentTask), создают новую (new TaskN()) при непустом image, устанавливают activeTask и showWindow2 = true, после чего сразу вызывают currentTask->prepare(ctx) – тяжёлые вычисления задачи
10. Формирование общего контекста задачи типа AppContext
11. Отрисовка окна задачи: при showWindow2 == true определяется заголовок окна через switch по activeTask, задаются позиция и размер окна. При manualDrawing == true запоминаются manualOrigin и manualArea, вызывается ImGui::Dummy для резервирования места и, при необходимости, t3->updatePreview(renderer) (через dynamic_cast) для актуализации превью Task3. При manualDrawing == false вызывается currentTask->drawByTexture(ctx) – виртуальный вызов полиморфной отрисовки через текстуры
12. Завершение кадра ImGui: ImGui::Render() формирует итоговый набор команд отрисовки
13. Отрисовка фона и интерфейса SDL-средствами: SDL_SetRenderDrawColor(30, 30, 30, 255) – тёмно-серый цвет фона, SDL_RenderClear – очистка буфера кадра, ImGui_ImplSDLRenderer3_RenderDrawData(ImGui::GetDrawData(), renderer) – вывод всех накопленных ImGui-примитивов через SDL_Renderer
14. Вывод кадра на экран: SDL_RenderPresent(renderer) – отправка содержимого буфера в окно; SDL_Delay(10) – задержка ~10 мс для снижения нагрузки на CPU
15. Завершение работы программы: после выхода из главного цикла (running == false) выполняется очистка ресурсов в обратном порядке относительно инициализации